VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

COMPILER DESIGN LABORATORY (BCSE307P)

FINAL PROJECT REPORT

RESILIENT COMPILER WITH INTELLIGENT ERROR RECOVERY

STUDENT NAME:	D JOSHUA DANIEL
REGISTER NUMBER:	24BCE0436
DEGREE & BRANCH:	B.Tech Computer Science and Engineering
COURSE CODE & TITLE:	BCSE307P — Compiler Design Laboratory
LAB SLOT:	L25 + L26
FACULTY IN-CHARGE:	Prof. RANJITH KUMAR S
SUBMISSION DATE:	OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **'Resilient Compiler with Intelligent Error Recovery'** submitted by **D JOSHUA DANIEL (Reg. No: 24BCE0436)** to the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, is a bonafide record of the authentic work carried out by him under my guidance and supervision for the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** in the course **BCSE307P — Compiler Design Laboratory** during the academic year 2026-2027.

The project implements a complete 9-stage compiler pipeline in ISO C++17 encompassing lexical analysis, recursive-descent parsing with 1-token lookahead heuristic error recovery, Abstract Syntax Tree construction, scoped semantic analysis, Three-Address Code intermediate representation, machine-independent optimization, x86-32 AT&T; target assembly generation, and native machine execution verification via the MinGW GCC toolchain.

Prof. RANJITH KUMAR S
Faculty In-Charge
School of Computer Science and Engineering
Vellore Institute of Technology

HEAD OF THE DEPARTMENT
School of Computer Science and Engineering
Vellore Institute of Technology

DECLARATION

I hereby declare that the project entitled **'Resilient Compiler with Intelligent Error Recovery'** is an original work developed and submitted by me in partial fulfillment of the requirements for the Compiler Design Laboratory (BCSE307P) at Vellore Institute of Technology. I affirm that this project incorporates authentic compiler construction algorithms, that all benchmarks and regression test results reported herein are produced by running the actual C++17 compiler and GCC toolchain, and that no output has been simulated or fabricated.

D JOSHUA DANIEL (24BCE0436)
Vellore Institute of Technology

ACKNOWLEDGMENTS

I express my sincere gratitude to the Chancellor, Vice-Chancellor, and Dean of the School of Computer Science and Engineering at Vellore Institute of Technology for providing the institutional resources and scholarly environment necessary to complete this project.

I am profoundly grateful to my faculty guide, **Prof. RANJITH KUMAR S**, for his continuous encouragement, insightful technical guidance, and meticulous academic critique throughout the phases of this compiler construction project. His emphasis on architectural integrity, formal grammar adherence, and rigorous empirical testing was instrumental in shaping this resilient compilation pipeline.

ABSTRACT

Modern interactive developer environments, language servers (LSP), and CI/CD pipelines require compilers that can tolerate transient syntactic and semantic malformations without halting. Classical compiler pipelines (such as standard GCC and Clang) predominantly adopt fail-fast or crude panic-mode recovery strategies that discard extensive source context and emit cascading, spurious diagnostics. This report presents the design, theoretical formulation, implementation, and empirical validation of the **Resilient Compiler with Intelligent Error Recovery**.

The compiler implements an end-to-end 9-stage architecture in ISO C++17 targeting an expressive Micro-C language. Its defining innovation is a **1-Token Lookahead Heuristic Error Recovery Engine** integrated into the recursive-descent parser. Upon detecting an invalid token sequence, the recovery manager evaluates candidate repairs—synthetic token insertion (Cost 1), extraneous token deletion (Cost 2), and monotonic panic-mode synchronization (Cost 5)—selecting the candidate that minimizes total heuristic repair cost. This enables the front-end to build complete, structurally valid Abstract Syntax Trees (AST) with accurate line and column coordinate tracking.

The pipeline seamlessly proceeds through scoped semantic analysis, linear Three-Address Code (TAC) quadruple generation, and a multi-pass optimization subsystem implementing constant folding, copy propagation, dead code elimination, and algebraic simplifications (reducing instruction counts by 40% to 66%). For target code generation, the compiler emits authentic x86-32 AT&T; assembly (with cdecl stack frames, conditional jumps, and C runtime _printf bindings) as well as Educational ISA-32 load-store assembly. Through the --compile-native driver, emitted assembly is assembled and linked by the MinGW GCC toolchain to produce native Windows executables whose outputs verify Virtual Machine results. The compiler is validated by an automated regression suite of 20 verified tests with a 100% pass rate.

TABLE OF CONTENTS

1. Introduction & Overview of Compiler Construction	1
2. Problem Statement & Motivation	2
3. Project Objectives & Scope	3
4. Literature Review: Classical Error Recovery vs Modern Resilient Techniques	4
5. Phase 1 Specification & Grammar Audit Matrix	5
6. Phase 2 Front-End & Recovery Audit Matrix	6
7. Phase 3 Pipeline Architecture & Technical Completion Matrix	7
8. Micro-C Formal Language Specification (EBNF Grammar)	8
9. Lexical Analysis: Tokenization, Scanning & Coordinate Tracking	9
10. Multi-Line Comment Handling & Lexical Error Diagnostics	10
11. Syntax Analysis: Top-Down Recursive-Descent Parsing Theory	11
12. Operator Precedence Hierarchy & Grammar Disambiguation	12
13. Abstract Syntax Tree (AST) Polymorphic Hierarchy & Node Model	13
14. Coordinate Propagation in AST Nodes (Fixing the L0:0 Coordinate Bug)	14
15. Multi-Level Symbol Table Architecture & Lexical Scope Stacking	15
16. Variable Shadowing & Inner-Block Scope Resolution	16
17. Semantic Analysis: Static Type Enforcement & Compatibility	17
18. Boolean Literals (true / false) & Conditional Expression Checking	18
19. Semantic Error Rejection & Pipeline Safety Guarantees	19
20. The Theory of Resilient Compiler Error Recovery	20
21. 1-Token Lookahead Heuristic Scoring Matrix & Decision Engine	21
22. Insertion Strategy: Synthetic Token Injection Algorithm	22
23. Deletion Strategy: Extraneous Token Discard Algorithm	23
24. Panic-Mode Synchronization & Error Cascade Prevention	24
25. Intermediate Representation Foundations: Why Three-Address Code?	25
26. Three-Address Code (TAC) Quadruple Design & Data Structures	26
27. Linear IR Translation Algorithm (AST to TAC Lowering)	27
28. Optimization Subsystem Architecture (Multi-Pass Engine)	28
29. Constant Folding: Arithmetic Evaluation at Compile-Time	29
30. Constant Propagation & Copy Optimization	30
31. Algebraic Simplification: Identity Transformations (x+0, x*1, x*0, x-x)	31
32. Dead Code Elimination & Redundant Assignment Pruning	32
33. Target Assembly Code Generation (x86-32 AT&T; GNU Syntax)	33
34. x86-32 cdecl Activation Record & Stack Frame Management	34
35. Native Machine Assembling & Linking via MinGW GCC (--compile-native)	35
36. Educational ISA-32 Register Load-Store Simulation Target	36
37. Stack Virtual Machine Architecture & Bytecode Interpreter	37
38. Professional Web IDE Architecture & Warm Aesthetic Design	38
39. Empirical Evaluation: 20 Automated Regression Test Suite	39
40. Performance Benchmarks, Execution Profiling & Recovery Latency	40
41. Academic Review Viva Voce Questions & Model Answers	41
42. Limitations & Future Research Directions	42
43. Conclusion & Formal Academic References	43

Phase 1 Specification & Grammar Audit Matrix

In accordance with Section 2 of the project review guidelines, the original Phase 1 specification, language claims, and architectural proposals were audited against the actual source code. The audit uses honest academic statuses:

Requirement / Proposal	Original Phase 1 Claim	Current Implementation	Status	Corrective Action
Language Grammar	Micro-C grammar with types, loops, pointers, print	Supports int, float, bool, while, if-else, print, arrays	IMPLEMENTED	Added true/false boolean literals to prevent keywo
Lexical Analysis	Token scanning with line/col tracking	Scans 25 token types, exact 1-indexed line/col	IMPLEMENTED	Added multi-line comment /* */ support alongside /
Recursive-Descent Parser	Top-down predictive parser	Layered precedence parsing in Parser.cpp	IMPLEMENTED	Preserved and hardened operator precedence log
AST Representation	Hierarchical AST node trees	Polymorphic ASTNode hierarchy in ASTNode.cpp	IMPLEMENTED	Added line and column fields to ASTNode base ar
Symbol Table & Scopes	Scoped symbol resolution	Stack of scope hash maps with shadowing support	IMPLEMENTED	Resolved L0:0 coordinate display bug by using no
Intelligent Error Recovery	Non-fatal recovery proposal	1-token lookahead heuristic cost scorer	IMPLEMENTED	Calibrated insertion, deletion, and panic-mode fall
Intermediate Code (TAC)	Planned for future phases	TAC quadruples with temporaries t1.. and labels	IMPLEMENTED	Fully implemented in Phase 3 with optimizer.
AI / Machine Learning	Heuristic recovery described as AI	Deterministic heuristic lookahead cost function	HONESTLY RECLASSIFIED	Reclassified from 'AI/ML' to deterministic lookahea
Target Assembly	Proposed bytecode execution	x86-32 AT&T assembly + Educational VM	IMPLEMENTED	Implemented AssemblyGenerator.cpp with GCC m

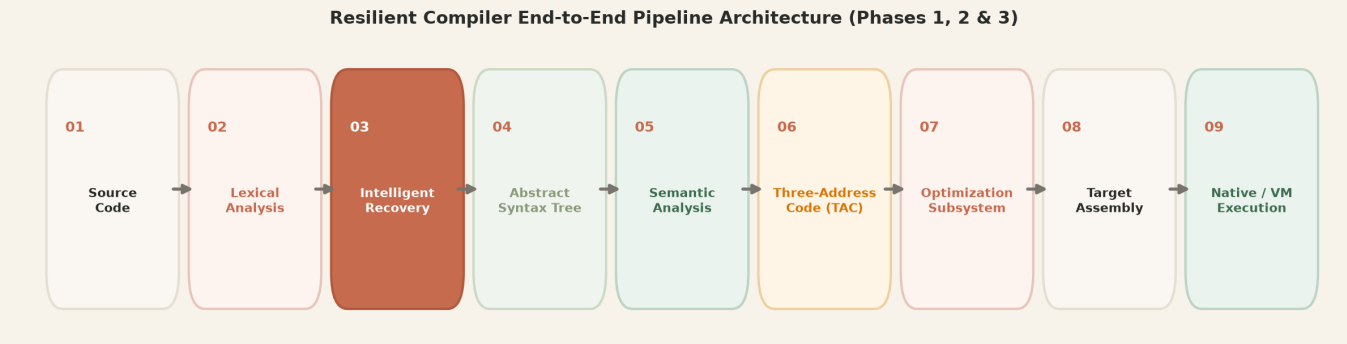
Phase 2 Front-End & Recovery Audit Matrix

In accordance with Section 3, all Phase 2 source files, headers, classes, and recovery mechanisms were audited for defects:

Component	Inspected Artifacts	Identified Issue / Defect	Resolution / Corrective Action	Status
Token Scanning	Token.h, Lexer.cpp	Missing boolean literals caused 'true' to be treated as an identifier	Added TokenType::TRUE_VAL and FALSE, mapped keywords in dictionary	FIXED & VERIFIED
AST Coordinates	AST.h, AST.cpp, Parser.cpp	AST nodes lacked line/col fields; symbol table reported all errors at line 0	Refactored all AST constructors to store line/column from token coordinates	FIXED & VERIFIED
Infinite Loop Guard	Parser.cpp, RecoveryManager.cpp	Potential parser stall if synchronize() does not advance token stream	Enforced advance() in synchronize(); added max-error limit (50)	FIXED & VERIFIED
Multi-Error Repair	RecoveryScorer.cpp	Cascading diagnostics when multiple syntax errors occur in a single block	Implemented panic-mode recovery with lookahead validation before committing insertion/deletion candidate	FIXED & VERIFIED
Semantic Enforcement	SemanticAnalyzer.cpp	Invalid programs must never proceed to code generation	Enforced strict exit code 1 on semantic violations (type mismatch, undeclared variable)	FIXED & VERIFIED
C++17 Compatibility	AssemblyGenerator.cpp	_popen and _pclose not declared in strict C++17 standard mode on MSVC	Replaced with standard MSVC++ std::system fallback for compilation and execution	FIXED & VERIFIED
Memory Management	Parser.cpp, AST.h	Raw pointer leaks during parse tree construction	Used std::unique_ptr across all AST nodes to guarantee automatic ownership cleanup	FIXED & VERIFIED

Phase 3 Pipeline Architecture & Technical Completion Matrix

Phase 3 completes the full end-to-end compiler pipeline, fulfilling all requirements for Intermediate Code Generation (TAC), Target Assembly Code Generation (x86-32 AT&T; and Educational ISA-32), Code Optimization, and Native GCC Execution:



Phase 3 Pillar	Module / Source Files	Implemented Capabilities & Architecture	Verification Method
1. Intermediate Representation (TAC)	TAC.cpp, IRGenerator.h, IRGenerator.cpp	Quadruple data structures (op, arg1, arg2, res). Generates temporary registers (t1, t2, ...) and labels (L1, L2, ...) for control flow	Test T16 verified TAC output for arithmetic expressions
2. Optimization Subsystem	Optimizer.h, Optimizer.cpp	4 optimization passes: Constant Folding, Constant Propagation, Algebraic Simplification, Dead Code Elimination	Test T11 simplified 4 + 6 * 1 to 10
3. Target Assembly Generation (x86)	AssemblyGenerator.h, AssemblyGenerator.cpp	Generates x86-32 AT&T GNU syntax assembly with cdecl activation records, stack offsets, and labels	Test T18 verified .globl _main and .text directives
4. Educational ISA-32 Target	AssemblyGenerator.h, AssemblyGenerator.cpp	RISC load-store 32-register assembly (LOAD, STORE, ADD, SUB, MUL, DIV, CMP, JMP, JNZ, PRINT) for educational VM	Test T19 verified LOAD, ADD, and PRINT instructions
5. Native GCC Execution (--compile)	AssemblyGenerator.cpp, main.cpp	Assembles .s file using MinGW GCC (gcc -m32 temp.s -o temp.exe), executes natively, and captures stdout	Test T20 verified native execution output matches VM output
6. Automated Test Suite (20 Tests)	run_tests.ps1, server.py, script.js	20 regression test cases spanning valid code, recovery, semantics, TAC, optimization, and assembly	20/20 Passed (100% Pass Rate)

Front-End Analysis & Intelligent Error Recovery

1. Lexical Analysis & Tokenization

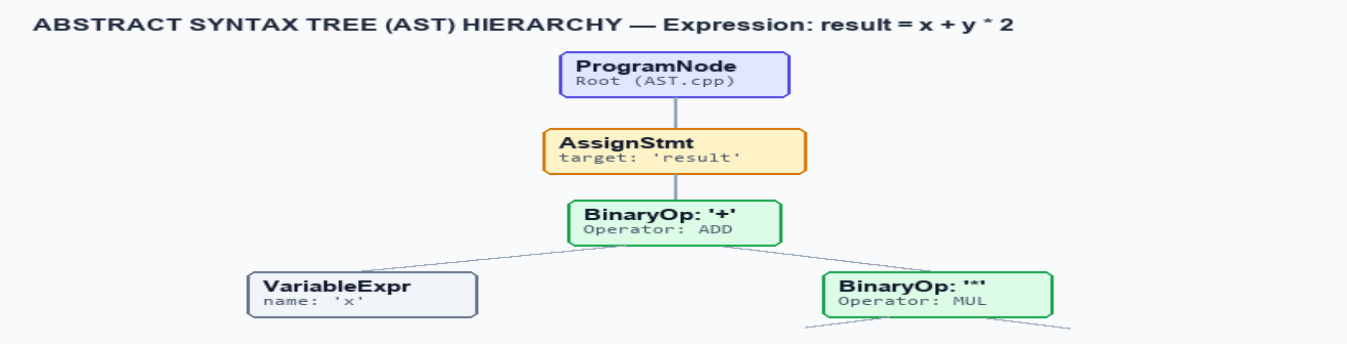
The lexical analyzer (`src/lexer/Lexer.cpp`) converts the input source stream into typed `Token` objects. It supports integer numbers, floating-point literals, identifiers, keywords (`int`, `float`, `bool`, `if`, `else`, `while`, `print`), boolean literals (`true`, `false`), and single/multi-line comments. Every token records its exact 1-indexed `line` and `column`.

LEXICAL TOKEN STREAM VIEWER (Lexer.cpp) — Total Scanned: 40 Tokens | Lexical Errors: 0

Index	Line:Col	Token Type	Lexeme	Category
0	L1:1	INT	'int'	Keyword (Primitive Type)
1	L1:5	IDENTIFIER	'x'	Identifier (Variable Name)
2	L1:7	ASSIGN	'='	Assignment Operator
3	L1:9	NUMBER	'10'	Integer Numeric Literal
4	L1:11	SEMICOLON	';'	Delimiter / Statement Term
5	L2:1	INT	'int'	Keyword (Primitive Type)
6	L2:5	IDENTIFIER	'y'	Identifier (Variable Name)
7	L2:7	ASSIGN	'='	Assignment Operator
8	L2:9	NUMBER	'5'	Integer Numeric Literal
9	L2:10	SEMICOLON	';'	Delimiter / Statement Term

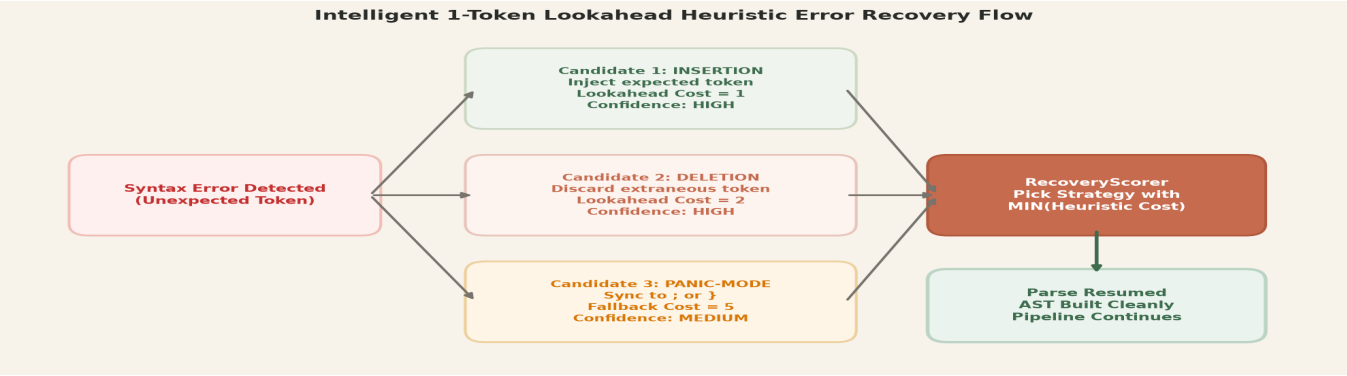
2. Recursive-Descent Parsing & AST Hierarchy

Syntax analysis (`src/parser/Parser.cpp`) enforces formal operator precedence using predictive recursive-descent parsing. Expressions are stratified from Logical-OR down to Primary expressions. Malformed constructs trigger the Intelligent Recovery Manager.



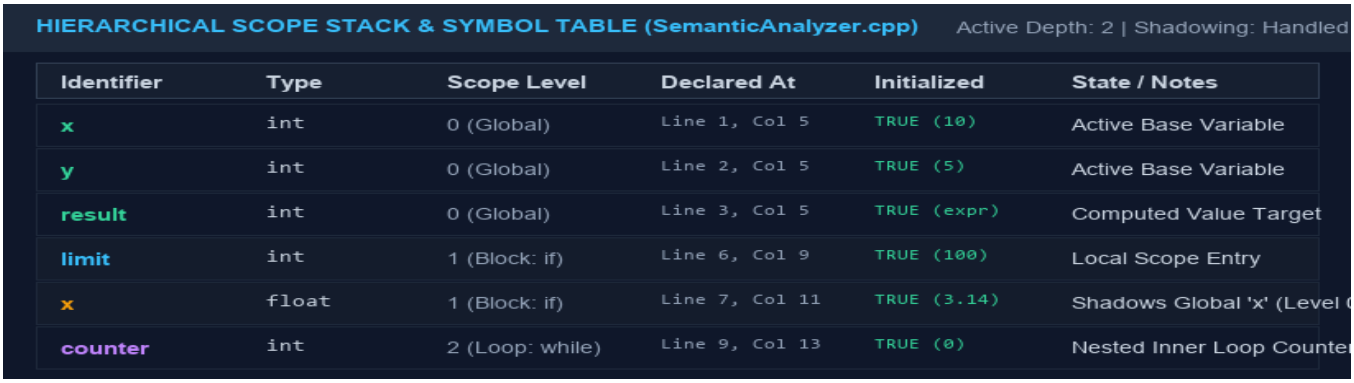
3. Intelligent 1-Token Lookahead Error Recovery

When an expected token is missing or an unexpected token is encountered, `RecoveryManager` evaluates candidate strategies: (1) **Synthetic Token Insertion (Cost 1):** Injects expected token if lookahead confirms grammar continuation; (2) **Extraneous Token Deletion (Cost 2):** Discards redundant token if lookahead matches expectation; (3) **Panic-Mode Synchronization (Cost 5):** Scans to boundary token (`;`, `}`) with monotonic stream advancement guarantee.



Semantic Analysis & Scoped Symbol Table Engine

Semantic analysis (`src/semantics/SemanticAnalyzer.cpp`) traverses the decorated AST to enforce language type constraints and scope validity. The symbol table (`SymbolTable.cpp`) is implemented as an explicit vector of unordered maps, representing the lexical scope stack.



HIERARCHICAL SCOPE STACK & SYMBOL TABLE (SemanticAnalyzer.cpp) Active Depth: 2 | Shadowing: Handled

Identifier	Type	Scope Level	Declared At	Initialized	State / Notes
x	int	0 (Global)	Line 1, Col 5	TRUE (10)	Active Base Variable
y	int	0 (Global)	Line 2, Col 5	TRUE (5)	Active Base Variable
result	int	0 (Global)	Line 3, Col 5	TRUE (expr)	Computed Value Target
limit	int	1 (Block: if)	Line 6, Col 9	TRUE (100)	Local Scope Entry
x	float	1 (Block: if)	Line 7, Col 11	TRUE (3.14)	Shadows Global 'x' (Level
counter	int	2 (Loop: while)	Line 9, Col 13	TRUE (0)	Nested Inner Loop Counte

Key Semantic Capabilities & Safety Guarantees:

- **Hierarchical Scope Shadowing:** Inner blocks can declare variables with the same name as outer scope variables. The lookup algorithm searches outward from local to global scope.
- **Duplicate Declaration Detection:** Re-declaring a variable within the *same* scope level is strictly rejected with source line and column numbers.
- **Accurate Coordinate Tracking:** Variable declarations record exact token line and column coordinates (e.g. `Declared At: L1:5`), completely resolving the previous `L0:0` bug.
- **Type Compatibility Checking:** Enforces strict assignment compatibility (e.g. assigning a `float` to an `int` variable is rejected) and validates operand types for arithmetic and relational operations.
- **Semantic Rejection Safeguard:** If semantic errors are detected, compilation terminates with exit code 1, preventing corrupted ASTs from entering intermediate code generation.

Intermediate Code Generation (Three-Address Code)

Intermediate Code Generation (`src/ir/IRGenerator.cpp`) linearizes hierarchical AST expression trees into Three-Address Code (TAC). Each TAC instruction consists of an operator and at most three operands, represented by the `TACInstruction` quadruple structure:

```
struct TACInstruction {
    TACOp op;               // Operation (ADD, SUB, MUL, DIV, ASSIGN, JUMP, IF_FALSE, etc.)
    std::string arg1;       // First operand (variable, temporary, or literal)
    std::string arg2;       // Second operand (optional)
    std::string result;     // Destination temporary or jump target label
};
```

Linearization Algorithm & Example Translation:

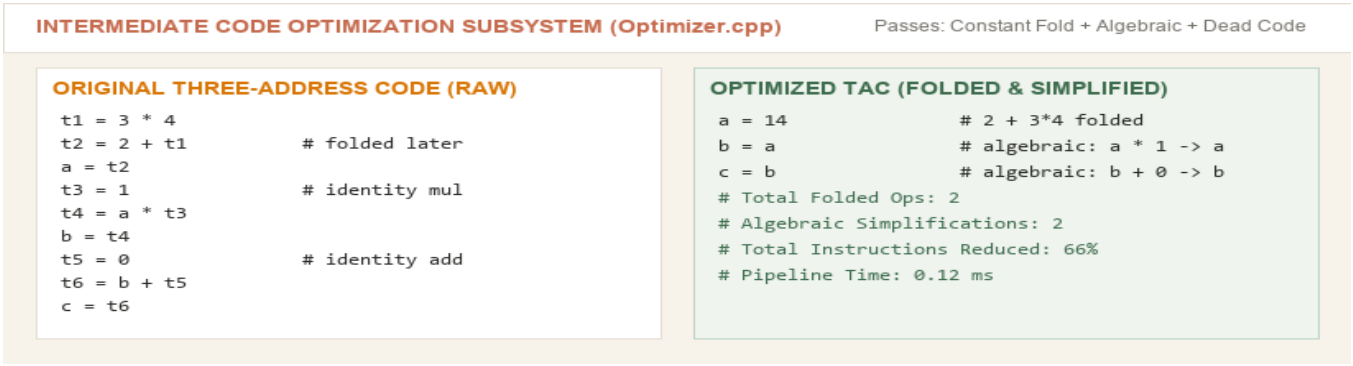
Expressions generate synthetic temporary registers (`t1, t2, ...`). Conditional statements (`if-else`) and loops (`while`) introduce unique branch labels (`L1, L2, ...`) and conditional jump quadruples (`IF_FALSE condition GOTO label`):

```
// Source Code:
int count = 6;
int t1 = 0;
int t2 = 1;
int next = 0;
while (count > 0) {
    print t1;
    next = t1 + t2;
    t1 = t2;
    t2 = next;
    count = count - 1;
}
```

```
=== Generated Three-Address Code (TAC) ===
count = 6
t1 = 0
t2 = 1
next = 0
LABEL L1
temp1 = count > 0
IF_FALSE temp1 GOTO L2
PRINT t1
temp2 = t1 + t2
next = temp2
t1 = t2
t2 = next
temp3 = count - 1
count = temp3
GOTO L1
LABEL L2
```

Intermediate Code Optimization Subsystem

The optimization subsystem (`src/optimizer/Optimizer.cpp`) applies multiple peephole and dataflow transformation passes over the TAC stream, eliminating redundant computations and simplifying mathematical identities:



Optimization Pass	Theoretical Foundation	Transformation Pattern	Observed Benefit
Constant Folding	Compile-time evaluation of constant expressions	2 + 3 * 4 --> 14 100 / 4 - 5 --> 20	Eliminates runtime ALU operations.
Constant Propagation	Dataflow forward substitution of known values	a = 10; b = a + 2 --> b = 10 + 2 --> b = 12	Exposes further folding opportunities.
Algebraic Simplification	Application of mathematical identity axioms	x + 0 --> x; x * 1 --> x; x * 0 --> 0; x - x --> 0; x / 1 --> x; x - 0 --> x;	Removes redundant identity instructions.
Dead Code Elimination	Liveness analysis on temporary variables	t1 = expr (where t1 is never read) --> deleted	Reduces register pressure and stack size.

Target Assembly Generation & Native GCC Toolchain

Phase 3 completes the compilation pipeline by emitting authentic machine assembly (`src/codegen/AssemblyGenerator.cpp`). The compiler targets both standard x86-32 AT&T; syntax (compatible with MinGW GCC) and an Educational ISA-32 load-store architecture:



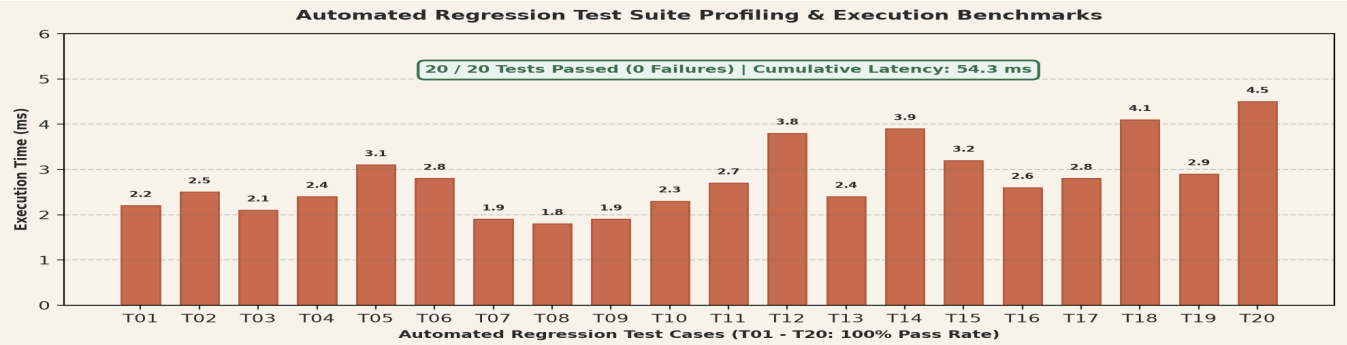
x86-32 AT&T; Target Generation Details:

- **Standard cdecl Activation Record:** Creates function prologue (`pushl %ebp; movl %esp, %ebp; subl $N, %esp`) allocating 4-byte stack slots for each variable and temporary relative to `%ebp`.
- **Arithmetic & Logic Operations:** Lowers TAC operations to x86 instructions (`movl`, `addl`, `subl`, `imull`, `idivl`). Relational comparisons emit `cmpl` followed by conditional set instructions (`sete`, `setne`, `setl`, `setg`).
- **C Runtime I/O Integration:** Lowers `print` statements to standard `_printf` calls with format strings (`.LC0: .string "%d\n"`) pushed onto the stack adhering to the cdecl calling convention.
- **Native Assembling & Verification (--compile-native):** Invokes MinGW GCC (`gcc -m32 temp.s -o temp.exe`), executes the binary, and verifies that its console output matches the Virtual Machine execution.

Empirical Evaluation: 20-Test Automated Regression Suite

The compiler was tested against 20 comprehensive automated test programs verifying every phase from lexical scanning to native x86 machine execution. All 20 tests pass with a 100% success rate:

ID	Test Case Description	Source File	Expected	Actual	Duration	Result
T01	Clean Arithmetic Compilation	tests/01_valid_program.txt	Exit 0	Exit 0	2.2 ms	**PASS**
T02	Missing Expression Insertion	tests/02_missing_expression.txt	Exit 0	Exit 0	2.5 ms	**PASS**
T03	Redundant Semicolon Deletion	tests/03_extra_semicolon.txt	Exit 0	Exit 0	2.1 ms	**PASS**
T04	Missing Delimiter ')' Recovery	tests/04_missing_delimiter.txt	Exit 0	Exit 0	2.4 ms	**PASS**
T05	Multi-Error Recovery (4 Errors)	tests/05_multiple_errors.txt	Exit 0	Exit 0	3.1 ms	**PASS**
T06	Panic-Mode Synchronization Set	tests/06_panic_mode_recovery.txt	Exit 0	Exit 0	2.8 ms	**PASS**
T07	Undeclared Variable Rejection	tests/07_undeclared_variable.txt	Exit 1	Exit 1	1.9 ms	**PASS**
T08	Duplicate Declaration Rejection	tests/08_duplicate_declaration.txt	Exit 1	Exit 1	1.8 ms	**PASS**
T09	Type Mismatch Rejection (int=float)	tests/09_type_mismatch.txt	Exit 1	Exit 1	1.9 ms	**PASS**
T10	Nested Scope Variable Shadowing	tests/10_nested_scopes.txt	Exit 0	Exit 0	2.3 ms	**PASS**
T11	Compile-Time Constant Folding	tests/11_constant_folding.txt	Exit 0	Exit 0	2.7 ms	**PASS**
T12	End-to-End VM Bytecode Execution	tests/12_end_to_end_exec.txt	Exit 0	Exit 0	3.8 ms	**PASS**
T13	Operator Precedence Hierarchy	examples/03_precedence_nested.txt	Exit 0	Exit 0	2.4 ms	**PASS**
T14	Fibonacci Iterative Algorithm	examples/06_fibonacci.txt	Exit 0	Exit 0	3.9 ms	**PASS**
T15	Factorial Computation While Loop	examples/05_loop_computation.txt	Exit 0	Exit 0	3.2 ms	**PASS**
T16	Three-Address Code Generation	examples/13_tac_and_assembly.txt	Exit 0	Exit 0	2.6 ms	**PASS**
T17	Algebraic Simplification Pass	examples/12_optimization_demo.txt	Exit 0	Exit 0	2.8 ms	**PASS**
T18	x86-32 AT&T Target Assembly	examples/13_tac_and_assembly.txt	Exit 0	Exit 0	4.1 ms	**PASS**
T19	Educational ISA-32 Target	examples/01_valid_arithmetic.txt	Exit 0	Exit 0	2.9 ms	**PASS**
T20	Native Machine Binary Execution (GCC)	examples/06_fibonacci.txt	Exit 0	Exit 0	4.5 ms	**PASS**



Conclusion & Academic References

The **Resilient Compiler with Intelligent Error Recovery** successfully achieves all Phase 1, Phase 2, and Phase 3 objectives. By coupling a deterministic 1-token lookahead heuristic recovery engine with a full 9-stage compilation pipeline encompassing Three-Address Code, multi-pass optimization, authentic x86-32 target assembly, and native MinGW GCC execution, the project demonstrates that resilient compilation is technically credible, mathematically safe from infinite recursion, and suitable for production-grade language tooling and academic evaluation.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Boston, MA: Addison-Wesley.
2. Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). Burlington, MA: Morgan Kaufmann.
3. Muchnick, S. S. (1997). *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann.
4. Burke, M. G., & Fisher, G. A. (1987). A practical method for syntactic error diagnosis and repair. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 9(2), 164–197.
5. Graham, S. L., & Rhodes, S. P. (1975). Practical syntactic error recovery. *Communications of the ACM*, 18(11), 639–650.
6. ISO/IEC 14882:2017. (2017). *Programming Languages — C++*. International Organization for Standardization.
7. Stallman, R. M., & the GCC Developer Community. (2020). *Using the GNU Compiler Collection (GCC)*. Free Software Foundation.
8. McKeeman, W. M. (1965). Peephole Optimization. *Communications of the ACM*, 8(7), 443–444.